

Guía completa para ejecutar el pipeline TLC

Arquitectura Medallion con enfoque Kappa, procesamiento PySpark, modelo estrella Gold, modelos analíticos y consumo en Power BI.

FUENTE	PERIODO	SALIDA FINAL
NYC TLC Trip Record Data	2023-2026	Parquet Gold + Power BI

Repositorio analizado: Final-Gesti-n_De_Datos_Masivos
Documento generado a partir del código, configuraciones, notebooks y documentación disponibles en el proyecto.

0. Cómo usar esta guía

Esta guía explica qué resuelve el proyecto, cómo está organizado y qué debe ejecutarse para reconstruir el flujo completo. La ruta principal está pensada para Windows, Visual Studio Code, PySpark y Power BI Desktop.

Estado de esta copia del repositorio

Los datos pesados no están versionados en Git. En la revisión actual aparecen scripts, notebooks, metadatos y el PBIX, pero no todos los Parquet de viajes. Para reproducir el proyecto en otra computadora se debe ejecutar la ingesta o trasladar las carpetas de datos generadas.

Contenido

Bloque	Qué responde
1-3	Objetivo, arquitectura, datos y estructura del repositorio.
4-5	Preparación del entorno y ejecución rápida o completa.
6-7	Qué genera Silver, especialmente cleaned y normalización B1.
8-9	Modelo estrella Gold, modelos predictivos y notebooks.
10-12	Power BI, auditoría, errores frecuentes y lista final de verificación.

Decisión técnica vigente

El flujo implementado actualmente publica Parquet y los consume desde Power BI. MongoDB figura como compatibilidad opcional en requirements, pero no es necesario para ejecutar el pipeline vigente ni para conservar las relaciones del modelo estrella.

1. ¿De qué trata el proyecto?

El proyecto procesa registros masivos de movilidad publicados por la New York City Taxi and Limousine Commission. Integra viajes de Yellow Taxi, Green Taxi, For-Hire Vehicle (FHV) y High Volume For-Hire Vehicle (FHVHV), además del catálogo Taxi Zone Lookup.

El propósito no es solamente descargar archivos. El sistema controla su integridad, registra auditoría, detecta cambios de esquema, limpia y normaliza datos heterogéneos, prepara métricas de negocio, entrena modelos analíticos y entrega tablas listas para dashboards.

Objetivos funcionales

- Descargar archivos mensuales de forma idempotente, sin duplicar descargas válidas.
- Preservar el original en Raw y una copia controlada en Bronze.
- Medir calidad antes y después de la limpieza.
- Unificar nombres, tipos y significado de columnas entre servicios.
- Construir un modelo estrella agregado y eficiente para Power BI.
- Generar resultados de pronóstico, segmentación y clasificación.
- Mantener trazabilidad de ejecuciones, errores, reglas y artefactos.

Fuente oficial

NYC TLC Trip Record Data: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>. Los años 2023, 2024 y 2025 se tratan como periodos cerrados; 2026 se consulta dinámicamente y solo procesa los meses que ya estén publicados.

Flujo Medallion con procesamiento incremental tipo Kappa

Cada nuevo archivo sigue la misma ruta; no existe una rama batch paralela.



2. Arquitectura Medallion + Kappa

Medallion organiza la calidad del dato por capas. Kappa indica que cada archivo nuevo recorre el mismo camino de procesamiento; no se mantiene una segunda rama batch con lógica distinta. En este proyecto, la combinación facilita incorporar nuevos meses o el año 2026 sin rediseñar el pipeline.

Nivel	Responsabilidad	Salida principal
Fuente	Archivos oficiales Parquet y Taxi Zone Lookup CSV.	URLs públicas de NYC TLC.
Ingesta	Catálogo, descarga, validación de extensión, firma, hash y lectura Spark.	Manifest y archivos descargados.
Raw	Conserva la copia original sin transformación.	data/raw/
Bronze	Copia organizada y auditada por tipo, año y mes.	data/bronze/files/
Silver	Profiling, esquema, limpieza, enriquecimiento, normalización y calidad.	data/silver/cleaned y trip_data_normalized
Gold	Agregaciones de negocio, dimensiones y hechos.	data/gold/modelo_estrella/
ML	Pronóstico, segmentación y clasificación.	Tres hechos analíticos Parquet.
BI	Modelo semántico, medidas DAX y dashboards.	Dashboard_Final.modelos_Final.pbix

Auditoría transversal

La auditoría no es una capa posterior. Acompaña la ejecución completa y registra pipeline, etapas, reglas de calidad, archivos producidos, duración y mensajes de error.

Tipos de datos y diferencias esperadas

Yellow y Green contienen importes, distancia, pago y pasajeros con mayor frecuencia. FHV puede no incluir tarifa, propina o forma de pago; en esos casos los nulos son semánticamente válidos. La normalización conserva columnas comunes y agrega nulos tipados cuando una fuente no dispone de un atributo.

3. Mapa del repositorio

Ruta	Función
config/	Configuración base y definición de etapas del pipeline auditado.
src/ingestion/	Descarga y validación Raw/Bronze.
src/silver/	Procesos productivos de profiling, esquema, limpieza, normalización y calidad.
src/gold/	Construcción incremental del modelo estrella.
src/audit/	Auditoría, runner de comandos y vistas para Power BI.
notebooks/ingestion/	Versión didáctica de la ingesta.
notebooks/silver/	Visualizaciones de cada etapa y notebook ejecutable de normalización B1.
notebooks/gold/	Versión exploratoria del modelo estrella incremental.
notebooks/models/	Entrenamiento y publicación de los tres modelos analíticos.
data/	Datos y resultados generados localmente; no se versionan por tamaño.
audit/runtime/	Tablas históricas y vistas latest_* para el dashboard de auditoría.
dashboards/powerbi/	PBIX final y recursos del dashboard de auditoría.
tests/	Pruebas de auditoría, calidad y estructura Gold.
tools/hadoop/bin/	winutils.exe y hadoop.dll para escritura Spark en Windows.

Archivos que controlan el recorrido

config/audit_pipeline.yaml define el orden de etapas y sus salidas obligatorias. src/pipeline.py reemplaza variables como año y tipo de taxi, ejecuta cada comando, captura logs y detiene el flujo si una etapa requerida falla.

Regla operativa

Ejecute siempre los comandos desde la raíz del repositorio. Las rutas se resuelven con respecto a esa ubicación y varios notebooks buscan el proyecto recorriendo carpetas superiores.

4. Preparar la computadora

Requisitos

Componente	Recomendación	Comprobación
Python	3.11 o 3.12 de 64 bits.	<code>python --version</code>
Java	JDK 17 de 64 bits para PySpark.	<code>java -version</code>
Memoria	8 GB mínimo; 16 GB o más para varios años.	Cerrar aplicaciones pesadas.
Disco	Decenas de GB para ejecución completa.	Revisar espacio antes de descargar.
VS Code	Extensiones Python y Jupyter.	Abrir y seleccionar kernel .venv.
Power BI	Power BI Desktop de 64 bits.	Abrir el PBIX del proyecto.

Crear el entorno virtual

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Si PowerShell bloquea la activación

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

Verificaciones mínimas

```
python --version
java -version
python -m src.pipeline --help
python src\ingestion\tlc_ingestion_nivel_2_3.py --help
pytest tests -v
```

Hadoop en Windows

No debe descargar winutils manualmente. El proyecto ya incluye `tools/hadoop/bin/winutils.exe` y `hadoop.dll`. Los scripts de normalización y Gold configuran `HADOOP_HOME` antes de iniciar Spark.

5. Ejecución recomendada del pipeline

La forma más segura es probar primero con un año y uno o dos tipos de taxi. Después se amplía el rango. El orquestador ejecuta ingesta, Silver, normalización y Gold en el orden definido, y registra todos los resultados.

5.1 Prueba pequeña

```
python -m src.pipeline --years 2026 --taxi yellow green --force-quality --live-output
```

Esta prueba descarga los meses de 2026 que estén disponibles. Si se desea controlar estrictamente la cantidad de archivos, puede ejecutarse primero el script de ingesta con `--max-files 4` y luego relanzar el pipeline con `--skip-ingestion`.

```
python src\ingestion\tlc_ingestion_nivel_2_3.py `
--years 2026 `
--trip-types yellow green `
--max-files 4 `
--workspace-dir data
```

```
python -m src.pipeline --years 2026 --taxi yellow green `
--skip-ingestion --force-quality --live-output
```

5.2 Ejecución histórica completa

```
python -m src.pipeline `
--years 2023 2024 2025 `
--taxi all `
--force-quality `
--live-output
```

5.3 Incorporar el año dinámico

```
python -m src.pipeline --years 2026 --taxi all --force-quality --live-output
```

Uso de memoria

No lance todos los años si todavía está validando el entorno. El proceso de limpieza tiene una configuración de Spark más exigente y la ejecución completa puede tardar horas. Procese por año cuando la memoria o el disco sean limitados.

6. Qué ocurre paso a paso

Etapa	Acción	Resultado
1. INGESTION	Construye URLs, descarga, valida firma Parquet, metadata, SHA-256 y lectura Spark.	data/raw; data/bronze/files; manifest JSONL
2. PROFILING INICIAL	Mide filas, columnas, nulos y calidad antes de transformar.	silver/profiling_inicial/_profiles
3. SCHEMA CHANGES	Compara estructuras entre años y detecta columnas o tipos cambiantes.	silver/schema_detection/_reports
4. UNIFIED SCHEMA	Define el esquema objetivo por tipo de servicio.	silver/unified_schema/_schemas
5. CLEANING	Tipa, valida, elimina inválidos, agrega lookup y columnas derivadas.	silver/cleaned//year=YYYY
6. NORMALIZATION B1	Mapea nombres heterogéneos a un esquema canónico ligero.	silver/trip_data_normalized
7. QUALITY	Aplica reglas de calidad sobre los datos cleaned y publica reportes.	silver/quality/_quality_reports
8. PROFILING FINAL	Compara la calidad final con el perfil Bronze.	silver/profiling_final/_profiles
9. GOLD	Agrega contribuciones, construye dimensiones/hechos y actualiza manifiesto.	gold/modelo_estrella

Reprocesar sin volver a descargar

```
python -m src.pipeline --years 2026 --taxi all --skip-ingestion --force-quality
```

Ejecutar solo una parte del flujo

```
python -m src.pipeline --years 2026 --taxi yellow `
  --skip-ingestion `
  --from-stage SILVER_04_BRONZE_TO_SILVER `
  --to-stage GOLD_MODELO_ESTRELLA `
  --force-quality --live-output
```

Los nombres de etapa deben coincidir exactamente con config/audit_pipeline.yaml. El límite final es inclusivo.

7. Qué sale de cleaned y de la normalización

7.1 Salida cleaned

La limpieza consolida los meses procesados de un tipo de taxi y un año. Aplica reglas de calidad, une el Taxi Zone Lookup y agrega atributos derivados. Cada dataset se escribe como una carpeta Spark con esta forma:

```
data/silver/cleaned/yellow/year=2025/yellow_2025_cleaned.parquet/  
  part-00000-....snappy.parquet  
  part-00001-....snappy.parquet  
  _SUCCESS
```

Una carpeta .parquet puede contener varios Parquet
Spark escribe un archivo part-* por partición. No son datasets duplicados: todos los part-* forman una sola tabla lógica. Power BI debe combinar únicamente los archivos .parquet internos.

7.2 Salida normalizada B1

B1 lee cleaned y conserva el mismo número de registros. Su tarea es homogeneizar columnas para que Gold pueda procesar todos los tipos de taxi con una sola lógica. Crea una salida por tipo y año:

```
data/silver/trip_data_normalized/  
  yellow_2025.parquet/  
  green_2025.parquet/  
  fhv_2025.parquet/  
  fhvhv_2025.parquet/
```

Aspecto	cleaned	trip_data_normalized
Propósito	Dato limpio, validado y enriquecido.	Contrato canónico común para Gold.
Granularidad	Un viaje por fila.	Un viaje por fila; no agrega ni elimina registros.
Columnas	Puede conservar más atributos específicos.	Orden canónico y extras esenciales.
Lookup	Incluye nombres de zonas cuando existen.	Conserva pickup/dropoff zone, borough y service zone.
Nulos	Dependen de la fuente original.	Puede crear nulos tipados para campos inexistentes.
Consumidor	Calidad y profiling final.	Modelo estrella Gold.

8. Modelo estrella Gold

Gold toma las entradas normalizadas y genera una tabla de hechos agregada. Mantiene un manifiesto de huellas por fuente; si un Parquet Silver no cambió, no vuelve a leerlo. Si una fuente cambia, reemplaza su contribución y evita duplicar viajes.



Salidas Gold

Grupo	Tablas
Dimensiones	dim_tiempo, dim_hora, dim_tipo_taxi, dim_zona_pickup, dim_zona_dropoff, dim_pago, dim_ratecode, dim_trip_type, dim_shared_ride, dim_proveedor
Hecho base	fact_viajes_agregados
Control	_control/process_manifest.json
Staging técnico	_staging/sources/fact_contribution.parquet
Auditoría	data/logs/gold_parquet_audit.jsonl

Granularidad de fact_viajes_agregados

Cada fila representa una combinación de mes, hora, tipo de taxi, zona de origen, zona de destino, pago, ratecode, tipo de viaje, viaje compartido y proveedor. Las métricas incluyen cantidad de viajes, totales y promedios de distancia, duración, montos y propina.

```
python src\gold\build_modelo_estrella.py --taxi all --years 2023 2024 2025
```

9. Notebooks: cuáles ejecutar y para qué

Notebook	Tipo	Uso
ingestion/tlc_nivel_2_3.ipynb	Procesa	Descarga y valida Raw/Bronze. Útil para demostración; el script es preferible en producción.
silver/01_profiling_inicial.ipynb	Visualiza	Lee perfiles JSON ya generados; no reemplaza el script.
silver/02_detect_schema_changes.ipynb	Visualiza	Muestra diferencias de columnas y tipos.
silver/03_generate_unified_schema.ipynb	Visualiza	Expone el esquema unificado.
silver/04_limpieza.ipynb	Visualiza	Muestra reportes y rutas cleaned; la transformación real está en el script.
silver/normalizacion_B1.ipynb	Procesa	Lee cleaned y escribe trip_data_normalized.
silver/05_analisis_de_calidad_de_datos.ipynb	Visualiza	Presenta resultados de reglas de calidad.
silver/06_profiling_final.ipynb	Visualiza	Compara perfiles iniciales y finales.
gold/modelo_estrella_parquet_powerbi.ipynb	Procesa	Versión exploratoria de Gold. No hace falta si el orquestador ya ejecutó Gold.
models/modelos_dashboards_powerbi.ipynb	Procesa	Ejecutar después de Gold para crear los tres hechos analíticos.

Ejecutar un notebook en VS Code

- Abra la carpeta raíz completa del proyecto, no solamente notebooks/.
- Active .venv y confirme que pyspark, ipykernel y jupyterlab están instalados.
- Abra el .ipynb y seleccione el kernel Python de .venv en la esquina superior derecha.
- Reinicie el kernel si hubo un error de memoria o murió el gateway Java.
- Ejecute Run All desde la primera celda; no empiece por una celda intermedia.
- Compruebe las rutas impresas y los conteos antes de continuar al siguiente notebook.

```
python -m ipykernel install --user `
--name tlc-pipeline `
--display-name "Python (TLC Pipeline)"
```

No ejecute Gold dos veces sin necesidad

El orquestador ya llama src/gold/build_modelo_estrella.py. Use el notebook Gold solo para explicar o explorar. Para el flujo normal, después del pipeline pase directamente al notebook de modelos.

10. Modelos analíticos

El notebook notebooks/models/modelos_dashboards_powerbi.ipynb consume fact_viajes_agregados y dim_tiempo. Filtra el periodo 2023-2025 antes de entrenar y publica tres tablas nuevas dentro de data/gold/modelo_estrella/hechos.

Modelo	Método	Qué responde	Salida
Pronóstico	RandomForestRegressor con rezagos 1, 2 y 3 meses.	¿Cuántos viajes se esperaban y cuál fue el error?	fact_pronostico_demanda.parquet
Segmentación	K-Means con k=4 y variables estandarizadas.	¿Qué zonas tienen perfiles de actividad semejantes?	fact_segmentacion_zonas.parquet
Clasificación	RandomForestClassifier.	¿La demanda será baja, media o alta por mes, hora, taxi y zona?	fact_clasificacion_demanda.parquet

División temporal

El pronóstico y la clasificación entrenan con 2023-2024 y evalúan 2025. Esta separación evita medir el modelo con los mismos datos usados para aprender. La segmentación construye perfiles de zona usando el rango válido completo.

Orden correcto

Paso	Acción
1	Actualizar Silver y Gold con el orquestador.
2	Ejecutar todas las celdas de modelos_dashboards_powerbi.ipynb.
3	Verificar que las tres salidas tengan filas y métricas válidas.
4	Actualizar Power BI.

Reconstrucción completa de modelos
Las tres tablas de resultados se sobrescriben en cada entrenamiento. Es correcto: un modelo nuevo puede cambiar predicciones o segmentos históricos.

11. Cargar y actualizar Power BI

Power BI debe apuntar a cada carpeta de tabla, no a la raíz modelo_estrella. Las carpetas _control y _staging son técnicas y no deben importarse.

Importar una tabla Parquet Spark

- Seleccione Obtener datos > Carpeta.
- Elija exactamente una carpeta, por ejemplo dimensiones/dim_tiempo.parquet.
- Seleccione Transformar datos.
- Filtre Extension para conservar solamente .parquet.
- Seleccione Combinar archivos y elimine Source.Name si no aporta al modelo.
- Asigne a la consulta el mismo nombre de la tabla.
- Repita con las dimensiones y hechos requeridos.

Relaciones

Todas las relaciones deben ser uno a muchos (1:*) y usar filtro simple desde la dimensión hacia la tabla de hechos. No relacione tablas de hechos entre sí. Las tablas predictivas comparten dim_tiempo, dim_hora, dim_tipo_taxi y dim_zona_pickup según sus claves disponibles.

Filtros recomendados

Filtro	Columna de dimensión
Año	dim_tiempo[anio]
Mes	dim_tiempo[nombre_mes], ordenado por dim_tiempo[mes]
Mes-año	dim_tiempo[etiqueta_mes], ordenado por dim_tiempo[orden_mes]
Tipo de taxi	dim_tipo_taxi[tipo_taxi]
Zona o borough	dim_zona_pickup o dim_zona_dropoff
Franja horaria	dim_hora[franja_horaria]
Forma de pago	dim_pago[forma_pago]

Actualizar

En modo Import, Power BI conserva los datos cargados aunque la consola esté cerrada. Para incorporar nuevos Parquet se debe ejecutar Actualizar. En Power BI Service, el Gateway y la computadora que aloja las carpetas deben estar encendidos durante la actualización programada.

12. Auditoría y comprobaciones

Después de una ejecución, el proyecto conserva auditoría histórica en audit/runtime y logs detallados en logs/pipeline. Las vistas latest_* alimentan el dashboard de control.

```
python -m src.audit.dashboard_views
```

```
Get-ChildItem .\audit\runtime\views\latest_*.parquet | `
  Select-Object Name, Length, LastWriteTime
```

Vista	Contenido
latest_pipeline_run.parquet	Estado y duración de la última ejecución.
latest_stage_runs.parquet	Detalle de cada etapa.
latest_quality_results.parquet	Reglas, métricas y fallos de calidad.
latest_file_runs.parquet	Archivos y artefactos detectados.

Comprobaciones rápidas

```
python src\silver\05_normalizacion_b1\normalizacion_b1.py `
  --taxi all --years 2026 --dry-run
```

```
python src\gold\build_modelo_estrella.py `
  --taxi all --years 2026 --dry-run
```

Dry-run no inicia Spark ni escribe datos. Sirve para confirmar que los datasets cleaned y normalizados tienen los nombres y rutas esperados.

13. Errores frecuentes y solución

Síntoma	Causa probable	Acción
JAVA_GATEWAY_EXITED	Java no está disponible, versión incompatible o el kernel conserva una sesión dañada.	Instalar/verificar JDK 17, cerrar sesiones Spark, reiniciar kernel y ejecutar desde la primera celda.
HADOOP_HOME unset	Spark intentó escribir antes de configurar utilidades Hadoop.	Usar los scripts actuales desde la raíz; confirmar tools/hadoop/bin. Reiniciar el kernel tras cambiar variables.
Java heap space	El driver se quedó sin memoria al contar, unir o escribir varias fuentes.	Procesar por año/tipo, cerrar Power BI, evitar cachés innecesarios y reiniciar kernel después de la caída.
Connection refused / Py4J	La JVM de Spark murió, normalmente después de OOM.	No continuar con el mismo objeto spark. Reiniciar kernel y volver a crear SparkSession.
Solo aparece yellow	Las otras fuentes fallaron, no existen en cleaned o no coinciden con el filtro de años.	Ejecutar B1 con --dry-run y revisar logs/silver_b_normalization_audit.jsonl.
Muchos nulos en FHV	La fuente FHV no publica todas las métricas de taxi amarillo/verde.	Validar pickup_datetime, zonas y bases; aceptar nulos no aplicables en tarifas o pagos.
Más/menos part-*.parquet	Spark controla el número de particiones según volumen y operación.	Interpretar la carpeta como una tabla. No comparar calidad por cantidad de partes.
Power BI no muestra meses	El segmentador usa una fecha o texto de la tabla de hechos sin orden.	Usar dim_tiempo[nombre_mes] y ordenar por mes; para varios años usar etiqueta_mes y orden_mes.
No hay datos en esta copia	Los datos y resultados están ignorados por Git.	Ejecutar ingesta/pipeline o copiar data/ desde una ejecución válida.

Después de un OutOfMemoryError

El error de Py4J que aparece después suele ser una consecuencia, no la causa original. La JVM ya terminó; reiniciar el kernel es obligatorio antes de reintentar.

14. Checklist final

- [] Estoy ubicado en la raíz del repositorio.
- [] .env está activo y requirements.txt fue instalado.
- [] python --version y java -version responden correctamente.
- [] Hay espacio en disco para Raw, Bronze, Silver y Gold.
- [] La ingesta generó Raw, Bronze y el Taxi Zone Lookup.
- [] cleaned contiene una carpeta por tipo y año procesado.
- [] B1 generó trip_data_normalized y conservó el conteo de filas.
- [] Quality y profiling final produjeron JSON no vacíos.
- [] Gold publicó diez dimensiones y fact_viajes_agregados.
- [] El notebook de modelos creó tres hechos analíticos.
- [] Las vistas latest_* de auditoría existen.
- [] Power BI usa relaciones 1:* desde dimensiones hacia hechos.
- [] Los meses y años se filtran desde dim_tiempo.
- [] Actualicé Power BI después de regenerar Parquet.

Secuencia mínima para una demostración

```
# 1. Pipeline hasta Gold
python -m src.pipeline --years 2026 --taxi yellow green `
  --force-quality --live-output

# 2. Abrir y ejecutar el notebook de modelos
jupyter lab notebooks\models\modelos_dashboards_powerbi.ipynb

# 3. Regenerar vistas de auditoría
python -m src.audit.dashboard_views

# 4. Abrir Power BI y seleccionar Inicio > Actualizar
```

Resultado final esperado

Un flujo reproducible desde archivos oficiales TLC hasta un modelo estrella y tablas predictivas en Parquet, con trazabilidad completa y dashboards actualizables en Power BI.

15. Referencias internas del proyecto

Tema	Archivo
Introducción y comandos	README.md
Orden de ejecución	config/audit_pipeline.yaml
Orquestador	src/pipeline.py
Normalización B1	src/silver/05_normalizacion_b1/normalizacion_b1.py
Modelo estrella	src/gold/build_modelo_estrella.py
Diseño Gold y Power BI	docs/modelo_estrella_parquet_powerbi.md
Modelos	notebooks/models/modelos_dashboards_powerbi.ipynb
Auditoría Power BI	dashboards/powerbi/audit/docs/
Pruebas	tests/

Esta guía fue construida revisando directamente el repositorio. Si el código o las rutas cambian, config/audit_pipeline.yaml y README.md deben actualizarse primero para mantener una única fuente de verdad.